



Resumo do Projeto: LibLink API

O projeto consiste em um ecossistema de micro serviços orquestrado por um **API Gateway**, focado na gestão de uma biblioteca comunitária. A arquitetura foi desenhada para demonstrar o desacoplamento, a transparência de rede e o nível máximo de maturidade REST através do uso de hipermídias.



Arquitetura Interna do Sistema

Simulamos um ambiente real de produção dividindo o software em três componentes independentes que se comunicam via HTTP:

1. **Books Service (Porta 8001):** Microserviço responsável pela persistência e manipulação do catálogo de livros. Ele permite consultas e alterações de status (PATCH) de forma isolada.
2. **Members Service (Porta 8002):** Microserviço que gerencia a base de dados de membros ativos, fornecendo informações sobre quem está apto a interagir com o sistema.
3. **API Gateway (Porta 8000):** O orquestrador central e único ponto de entrada para o cliente. Ele realiza a "composição" dos dados (buscando informações nos outros dois serviços) e injeta a inteligência de navegação.



Implementação de HATEOAS e RESTful

Atingimos a "Glória do REST" (Nível 3 de Maturidade) implementando **HATEOAS** (*Hypermedia as the Engine of Application State*) diretamente no Gateway:

- **Dinamismo de Estado:** As respostas JSON não contêm apenas dados brutos. Elas incluem um nó de **links** que informa ao cliente quais ações são permitidas naquele momento (ex: se um livro está disponível, a API oferece o link para "alugar"; se emprestado, oferece o link para "devolver").
- **Padronização:** Utilizamos verbos HTTP padronizados: **GET** para leitura, **POST** para ações disparadas pelo cliente e **PATCH** para atualizações parciais nos microserviços internos.



Cliente Web e Documentação

- **Web Client Interativo:** Desenvolvemos uma interface moderna em Tailwind CSS que consome o Gateway. O cliente é inteligente o suficiente para ler os links HATEOAS e renderizar botões de ação dinamicamente, sem caminhos "hardcoded".

- **Swagger (OpenAPI):** O Gateway fornece documentação automática e interativa. Através do endpoint `/docs`, é possível testar toda a arquitetura interna de forma visual, cumprindo o requisito de documentação profissional.
-

Checklist de Entrega (Regras do Projeto)

Certifique-se de validar cada item abaixo antes da apresentação:

- [x] **API Gateway:** Implementado como ponto único de entrada na porta 8000.
 - [x] **Conceito HATEOAS:** Links dinâmicos presentes nas respostas de `/gateway/livros` e `/gateway/dashboard`.
 - [x] **Documentação de API:** Swagger UI funcional e acessível no Gateway.
 - [x] **Mínimo de 2 APIs Internas:** Implementadas (Books Service e Members Service) simulando sistemas distribuídos.
 - [x] **Cliente Web:** Desenvolvido e funcional, acessível via rota `/client` no Gateway.
 - [x] **Estudo de Caso:** Sistema de biblioteca com fluxo de aluguel e devolução funcional.
 - [x] **GitHub:** Projeto versionado, com `.gitignore` e `README.md` explicativo.
-

Dicas para a Apresentação Presencial

1. **Mostre os Terminais:** Destaque que cada serviço roda em um processo/porta diferente, provando a distribuição.
2. **Prove o HATEOAS:** Alugue um livro no cliente e mostre no JSON (ou no log) que o link de ação mudou de "alugar" para "devolver" automaticamente.
3. **Tolerância a Falhas:** Desligue o terminal de Membros e mostre que o catálogo de livros continua funcionando, explicando o isolamento de falhas em microserviços.